

Relatório de Projeto Web

Relatório técnico-académico

Documento académico com registos textuais e visuais de commits, branches, merge com conflito, tags e preparação de colaboração GitHub.

Aluno	Sandro Ferreira Martins
Número	0133542
Email	Sandro.Martins.T0133542@edu.atec.pt
Tema	Git, GitHub, branches, merge, tags e registos visuais

Resumo

O trabalho demonstra a criação, evolução e análise de um repositório Git com foco em rigor operacional, documentação de registos e preparação para colaboração no GitHub. A estrutura inclui comandos essenciais, análise do histórico, resolução de conflito de merge, tagging semântica e material visual de suporte.

Palavras-chave

Git, GitHub, branches, merge, conflito, tags, documentação, registo visual

Registos principais	Screenshots SVG, report.md e logs de Git
Colaborador	vicsolucoes
Estado	Relatório revisto e pronto para submissão

Registos Visuais

As figuras seguintes sintetizam os momentos-chave do trabalho e substituem os blocos textuais de registo.

Figura 1. Estado inicial do repositório

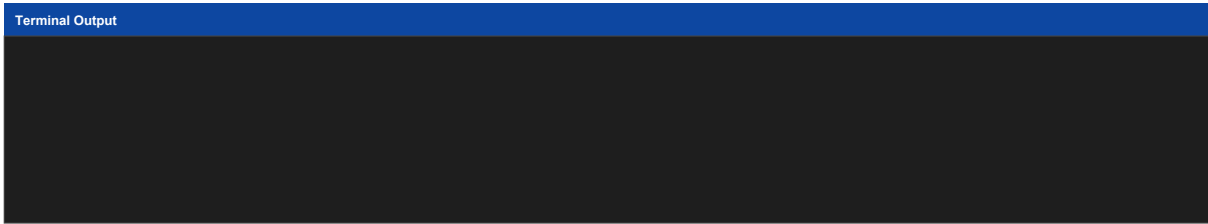


Figura 2. Estado do repositório durante o conflito

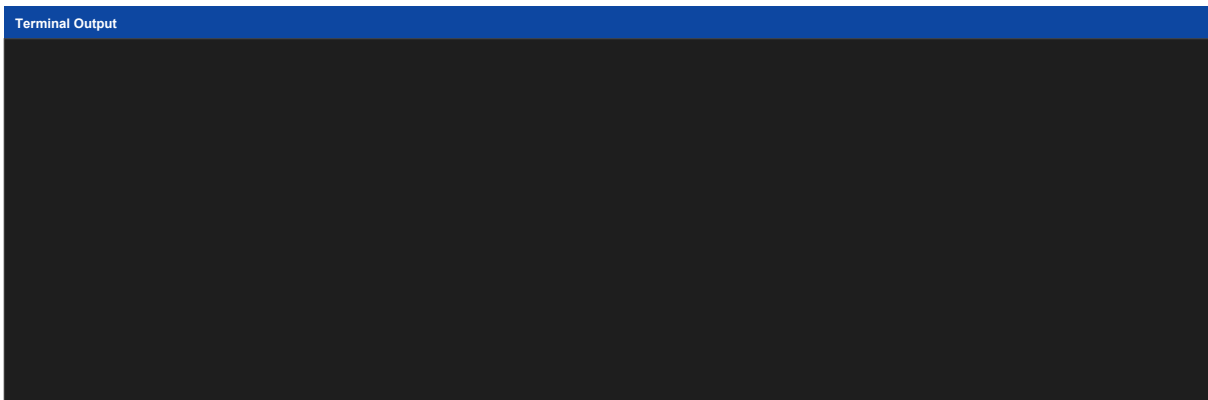


Figura 3. Diff do conflito com marcadores

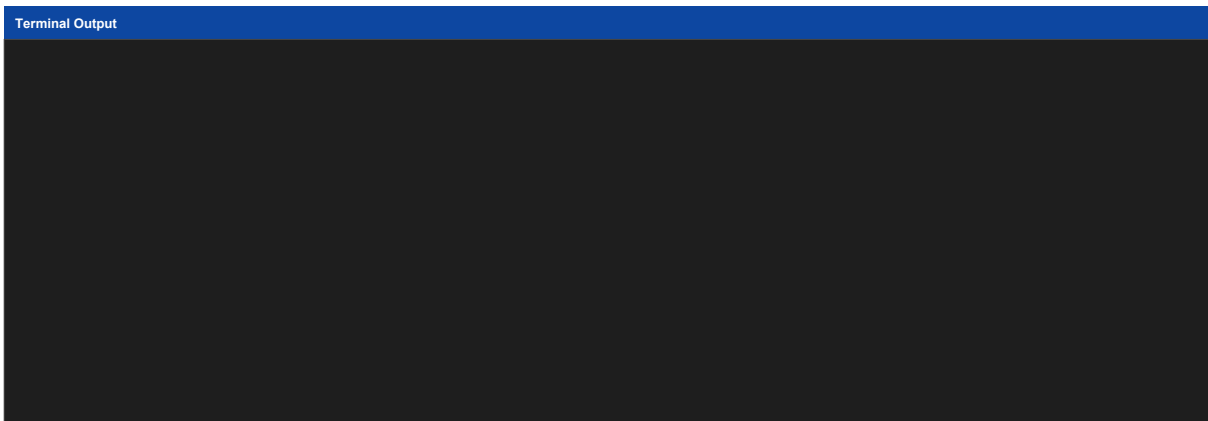


Figura 4. Histórico pós-merge em gráfico

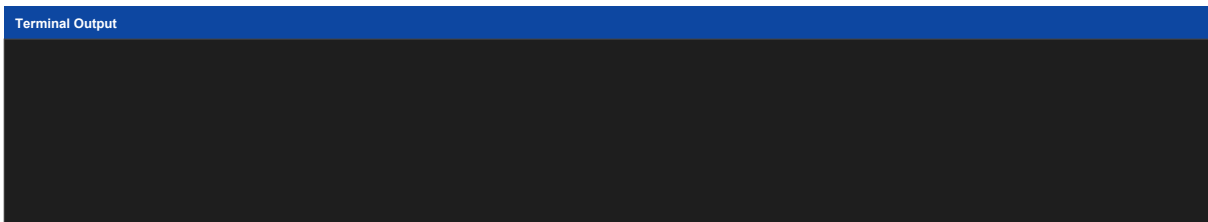
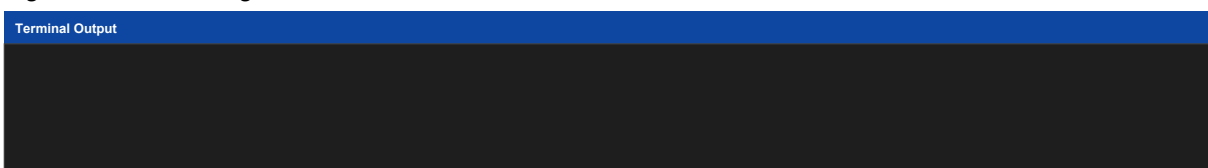


Figura 5. Lista de tags criadas



Email: Sandro.Martins.T0133542@edu.atec.pt

Objetivo

Demonstração prática e completa de conceitos de Git e GitHub através de um projeto web simples, incluindo:

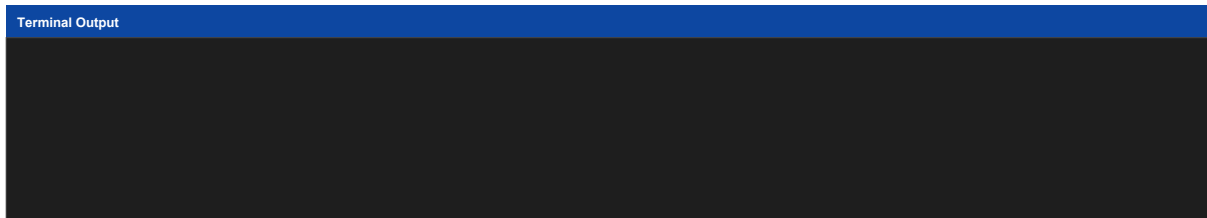
- Inicialização e configuração de repositório Git local
- Commits incrementais com mensagens descritivas
- Análise de repositório (status, diff, log)
- Gestão de branches (criação, merge)
- Resolução de conflitos intencional
- Versionamento com tags
- Preparação de registos visuais em screenshot/fotoreport
- Preparação da colaboração GitHub com colaborador externo

1. Inicialização do Repositório

Processo

```
git init
git config user.name "Sandro Ferreira Martins"
git config user.email "Sandro.Martins.T0133542@edu.atec.pt"
git status
```

Figura 1. Estado inicial do repositório



Registo textual: reports/01-git-status-clean.txt

2. Commits Incrementais

Commit 1: Documentação

```
git add README.md
git commit -m "docs: Add README with project description and instructions"
git show --stat --oneline HEAD
```

Hash: `bc7188c`

Commit 2: Estrutura Web

```
git add index.html styles.css script.js
git commit -m "feat: Add web skeleton (HTML, CSS, JavaScript)"
git status --short
```

Hash: `3800359`

Estrutura criada:

- `index.html` — Página HTML com identificação do aluno
- `styles.css` — Estilos CSS com gradiente e layout
- `script.js` — Script JavaScript que modifica o DOM

Commit 3: Funcionalidade A

```
git checkout -b feature/alunoA
# Modificação de index.html
git add index.html
git commit -m "feat(alunoA): Add feature A implementation"
git branch --show-current
```

Hash: `5354d65`

Commit 4: Funcionalidade B

```
git checkout -b feature/alunoB
# Modificação diferente de index.html
git add index.html
git commit -m "feat(alunoB): Add alternative feature B implementation"
git log --oneline --decorate -n 3
```

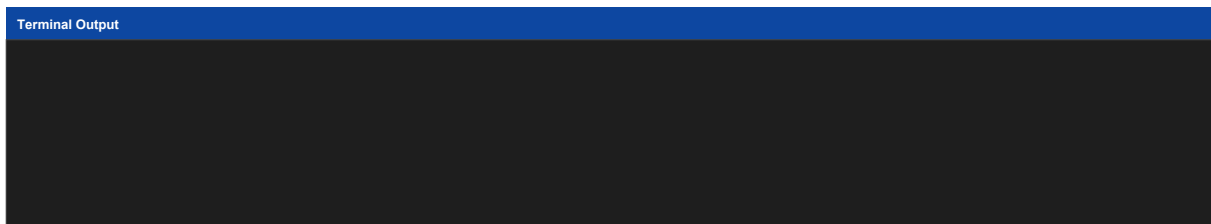
Hash: `513bce6`

3. Análise e Histórico

git status (Estado Inicial)

```
git status
git status --short
```

Figura 1. Estado inicial do repositório



Registro textual: [\[reports/01-git-status-clean.txt\]\(reports/01-git-status-clean.txt\)](#)

git log (Histórico Inicial)

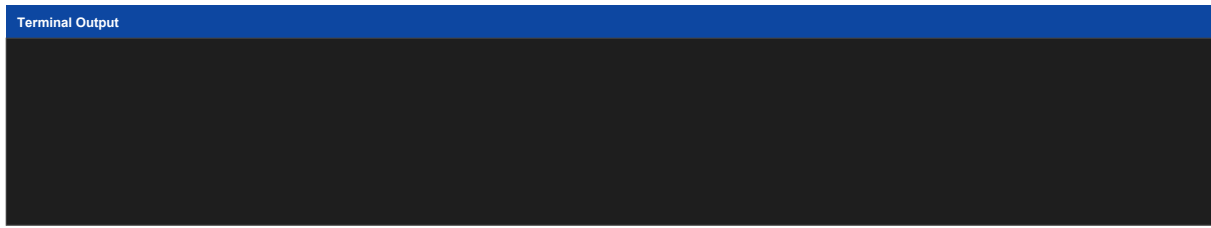
```
git log --oneline
git log --oneline --decorate --graph --all
```

Registro textual: [\[reports/02-git-log-initial.txt\]\(reports/02-git-log-initial.txt\)](#)

git log (Pós-Merge)

```
git log --oneline --graph --decorate --all
git show --summary --stat HEAD
```

Figura 4. Histórico pós-merge em gráfico



Registo textual: reports/06-git-log-postmerge.txt

4. Branches e Merge

Criação de Branches

1. `feature/alunoA` — Branch com implementação A
2. `feature/alunoB` — Branch com implementação B alternativa

```
git branch
git branch --all
git checkout feature/alunoA
git checkout feature/alunoB
```

Merge com Conflito Intencional

```
git checkout feature/alunoA
git merge feature/alunoB --no-ff
git status
git diff
```

****Resultado:**** Conflito detectado na mesma linha do ficheiro `index.html`

Registo do conflito:

Figura 2. Estado do repositório durante o conflito

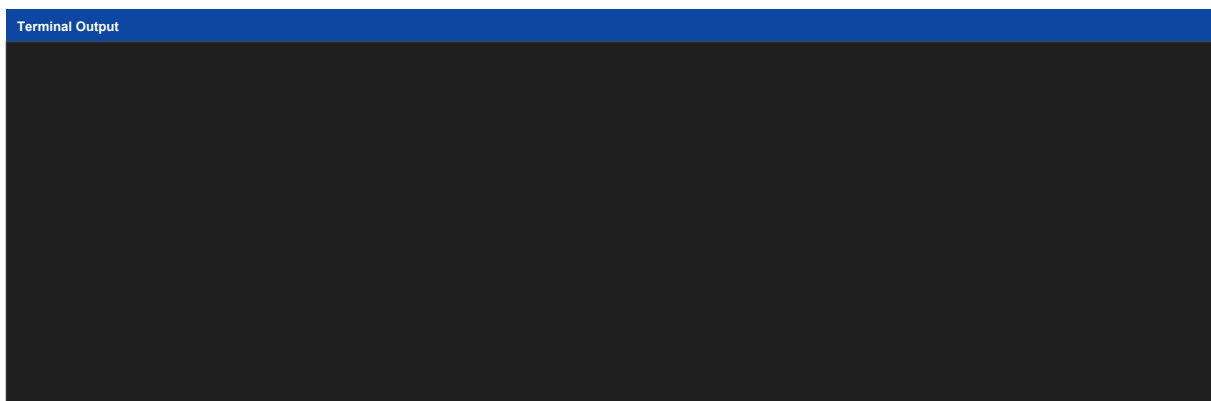
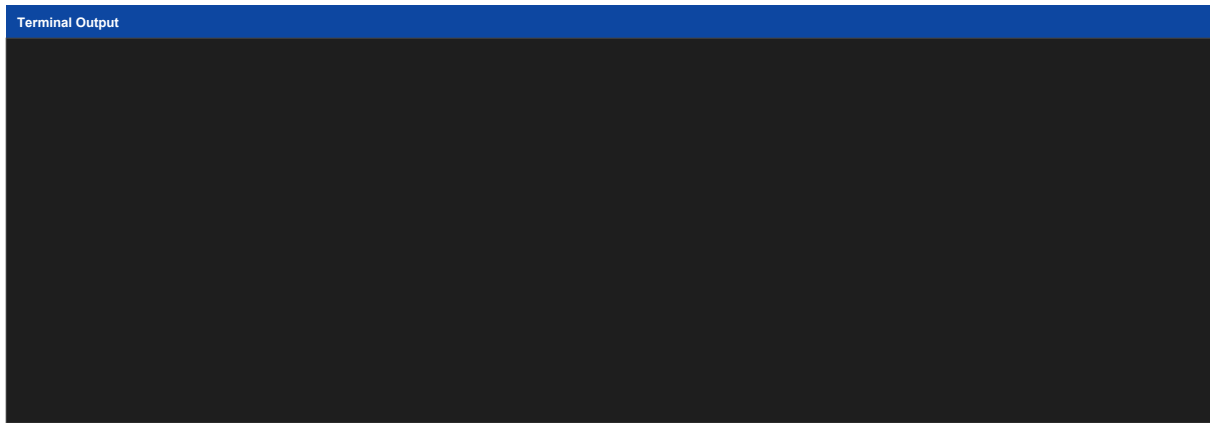


Figura 3. Diff do conflito com marcadores



- reports/03-merge-conflict-log.txt
- reports/04-git-status-conflict.txt
- reports/05-git-diff-conflict.txt

Marcadores de Conflito

```
<<<<<< HEAD
<p>
  <strong>Branch A:</strong> Implementação de funcionalidade específica para
    Aluno A
</p>
=====
<p>
  <strong>Branch B:</strong> Implementação alternativa de funcionalidade para
    Aluno B
</p>
>>>>>> feature/alunoB
```

Resolução Manual

Resolvido combinando as duas funcionalidades:

```
<p>
  <strong>Resolução:</strong> Funcionalidades de Aluno A e Aluno B integradas
    com sucesso!
</p>
```

Commit de resolução:

```
git add index.html
git commit -m "fix: Resolve merge conflict - integrate features A and B"
git log --oneline --graph --decorate -n 5
```

Hash: `1cdf758`

5. Tags e Versionamento

Tags Criadas

Tag Lightweight

```
git tag v1.0.0 -m "Version 1.0.0 - Initial release"
git tag --list
```

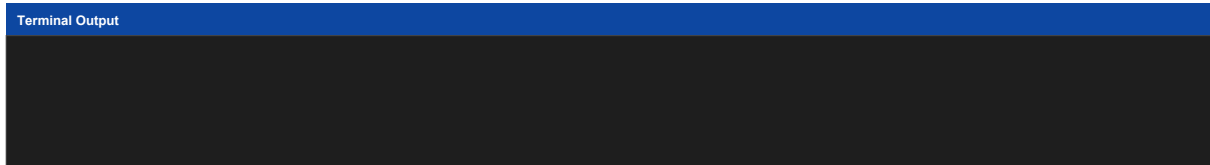
Tag Anotada

```
git tag -a v1.1.0 -m "Version 1.1.0 - After conflict resolution"
git show v1.1.0 --stat
```

Lista de tags:

```
v1.0.0  Version 1.0.0 - Initial release
v1.1.0  Version 1.1.0 - After conflict resolution
```

Figura 5. Lista de tags criadas



Registo textual: [\[reports/07-tags-list.txt\]\(reports/07-tags-list.txt\)](#)

6. Colaboração GitHub

Preparação do Remote

```
git remote add origin <URL_DO_REPOSITORIO>
git remote -v
git push -u origin main
git push -u origin feature/alunoA
git push -u origin feature/alunoB
git push origin --tags
```

Colaborador a Adicionar

No GitHub, adicionar o colaborador:

- `vicsolucoes` — perfil a convidar como collaborator do repositório

Registos de colaboração

```
git remote -v
git branch --all
git tag --list
```

7. Estrutura do Projeto Final

```
git-labs/
  ■■■ README.md
  ■■■ index.html
  ■■■ styles.css
  ■■■ script.js
  ■■■ reports/
    ■■■ report.md (este ficheiro)
    ■■■ 01-git-status-clean.txt
    ■■■ 02-git-log-initial.txt
    ■■■ 03-merge-conflict-log.txt
    ■■■ 04-git-status-conflict.txt
    ■■■ 05-git-diff-conflict.txt
    ■■■ 06-git-log-postmerge.txt
    ■■■ 07-tags-list.txt
    ■■■ screenshots/
      ■■■ git-status-initial.svg
      ■■■ git-status-conflict.svg
```

■■■ git-diff-conflict.svg
■■■ git-log-graph.svg
■■■ git-tags-list.svg

8. Como Executar Localmente

```
python -m http.server 8000
```

Abrir: `http://localhost:8000/index.html`

9. Conclusões

Este projeto demonstra corretamente:

- ■ Inicialização de repositório Git local com configuração de utilizador
- ■ Commits incrementais com mensagens claras e descritivas
- ■ Uso consciente de `git status`, `git diff`, `git log`
- ■ Criação de branches e merge com conflito intencional
- ■ Resolução manual de conflito e respetivo commit
- ■ Versionamento com tags (lightweight e anotada)
- ■ Documentação completa com registos textuais e visuais
- ■ Preparação para colaboração no GitHub com `vicsolucoes`

Relatório gerado automaticamente em 2026-05-04